ChatGPT

Building a Post-Android Phone OS on Existing Android Hardware

As of 6 May 2026, the answer is **yes, you can build your own phone OS for existing Android hardware**, but the sane route is **not** a clean-room rewrite from zero. The practical route is a staged migration: reuse the Android-era hardware plumbing you need at first, then replace pieces with mainline Linux as you gain control. On current commercial phones, the blocker is usually not the UI; it is the device-specific swamp underneath it: display, touch, modem, audio, camera, power management, verified boot, and updates. [1]

What is actually changing

It is **not accurate** to say that Android source is simply "closing in September." What is happening is more specific. Official AOSP documentation says `aosp-main` became read-only on March 27, 2025, recommends developers build against `android-latest-release`, and states that in 2026 Google will publish source to AOSP in **Q2 and Q4** rather than the old quarterly cadence. That makes life harder for downstream ROMs and independent platform work, but it does **not** mean public Android source disappears overnight. [2]

The separate September 2026 change is a developer-verification rollout for app installs on **certified Android devices** in **Brazil, Indonesia, Singapore, and Thailand** first, with broader rollout later. That is not a Europe-wide kill switch, and it is not a date on which existing phones in the Netherlands suddenly become unusable. [3]

The strongest evidence that Android itself is still very much alive in public is that AOSP continues to publish current security bulletins and corresponding source patches. Google's May 2026 bulletin was published on May 4, 2026 and states that corresponding AOSP source patches are released after publication. So the real shift is **less openness in day-to-day development and slower public source cadence**, not instant death of all post-stock Android work. [4]

What a phone OS actually has to replace

A phone OS is not just "Linux on ARM with a nice shell." On commercial smartphones, crucial parts are not open source and are often specific to the manufacturer and even the individual model. Canonical [5]-era and modern UBports [6] porting docs both say this explicitly for Ubuntu Touch ports, and they explain why ports often still rely on a device-specific vendor kernel and a minimal Android system to enable hardware. [7]

That is why projects such as Halium exist. The Halium project defines its job very narrowly: unify the hardware abstraction layer needed for GNU/Linux systems on Android phones by reusing the **vendor kernel**, **Android services required to talk to hardware**, and **libhybris**. It lists modem, camera, GPS, media codecs, and audio among the areas this layer has to bridge. [8]

Older Linux-phone porting guides make the same point from another angle. LuneOS, for example, describes a port as reusing the stock Android kernel, OpenGL HAL/drivers, audio/media HAL, and RIL

daemon for modem support, then layering a Linux userland on top. Its porting guide breaks the job into building Android pieces, kernel modifications, board support layers, and OS integration. That is a pretty honest x-ray of the actual workload. [9]

On top of that, you still need release engineering: bootloader behavior, signing, verified boot, rollback protection, OTA generation, and fault-tolerant updates. Google's own docs on GKI, AVB, and A/B updates show how much infrastructure a modern phone OS needs before it stops being a dev toy and starts being a safe device. GKI exists specifically to reduce kernel fragmentation by moving board/SoC specifics into vendor modules; AVB signs and verifies partitions; A/B updates keep a fallback slot so a bad OTA does not brick the phone. [10]

One more thing that trips people up: **Plasma Mobile is not a base OS**. KDE [11] describes Plasma Mobile as an open-source **user interface** that works with various Linux distributions. So "Plasma Mobile versus Phosh versus Lomiri" is a shell/UI choice, not the same thing as choosing your underlying base system. [12]

The five strongest base paths

The difficulty labels below are my synthesis from the current architecture of each project and the amount of hardware adaptation they still require.

1. **Droidian** — This is the most pragmatic Linux-first launchpad if your target is **phones that originally shipped with Android**. Its own docs say the goal is to run Debian on Android phones, using libhybris and Halium. Official project updates show real attention to phone essentials on official devices: fingerprint, NFC, MTP, camera work, hardware video playback, and integrated Waydroid management. Difficulty: **hard** on a phone that already has Halium work behind it; **extreme** on a totally unsupported device. Post-Android replacement potential: **medium-high** for enthusiasts, because it gives you a real Debian-style stack while still acknowledging the Android hardware reality instead of pretending it does not exist. [13]

2. **postmarketOS** — This is the strongest **long-term** route if your actual dream is "a real Linux phone OS, not Android with a different launcher." The project is explicit about its mission: extend device life with shared open components instead of device-specific dead ends. It uses Alpine because the base is tiny and because `pmbootstrap` gives a consistent cross-distro build/install workflow. Its current device list tracks **731 supported devices**, which makes it the broadest experimentation field in this space, but the project also says plainly that the current releases are aimed mainly at Linux enthusiasts and are not yet at Android/iOS polish. Difficulty: **medium-hard** on a device with existing support and a good upstream path; **very hard** on a fresh unsupported phone. Post-Android replacement potential: **highest of the five**, because it is the clearest road away from Android-era dependency and toward generic kernels and upstream maintenance. [14]

3. **Ubuntu Touch** — This is the strongest "mobile UX first" Linux base among the open projects for supported devices. Its supported-device site says it is mostly stable but still missing some features, and it has a broad list of community-supported Android handsets alongside Linux-native devices. The porting docs are unusually clear, and the project documents both the Halium route and the limitations that come from a read-only OTA-oriented root filesystem. It also supports Android apps through Waydroid, but that is an extra compatibility story rather than its native model. Difficulty: **hard**. Post-Android replacement potential: **medium**, because it is a good

usable Linux phone environment today, but it is less of a “strip it down and apt your way through the whole rootfs” hacker base than plain Debian/Alpine routes. [15]

4. **Sailfish OS** — From Jolla [16], this is the most commercially polished non-Android-like route in the list, especially on supported Sony [17] Xperia hardware. Jolla’s current support table shows live support on the Xperia 10 line and the Jolla C2, and its product pages emphasize an Android app layer for Linux platforms. The catch is that Android AppSupport requires a Sailfish license, the Play Store client and Google Play Services are not supported, and Jolla does not guarantee full Android app compatibility. Difficulty: **medium** on officially supported Sony devices; **very hard** outside that lane. Post-Android replacement potential: **medium-high** if you control hardware choices and accept some proprietary/licensed pieces; **lower** if your goal is a fully open community ecosystem that anyone can fork freely. [18]

5. **Mobian** — This is the cleanest “plain mobile Debian” reference base. The project site calls it a Debian derivative for mobile devices, and the Debian wiki says its goal is to bring Debian to mobile devices while minimizing Mobian-specific changes by upstreaming them; the same wiki also says Mobian primarily targets **Linux-first** mobile devices. That makes it excellent as a conceptual base if what you want is a stripped-down Linux phone distro with normal package management and a relatively desktop-Linux mindset. It makes it **less** ideal as the primary bring-up base for random Android phones, because on those devices you usually end up needing exactly the sort of Android adaptation work that led hybris-mobian to become Droidian. Difficulty: **very hard** on generic Android hardware. Post-Android replacement potential: **high on Linux-native hardware, lower on legacy Android handsets**. [19]

Banking, identity, and app compatibility

This is the bit where the dream hits the wall. Linux-first phone OSes can run Android apps only by adding a compatibility layer. Ubuntu Touch documents Waydroid for Android apps. Waydroid itself says it runs a full Android system in a container on GNU/Linux, and that its runtime currently ships with an Android 11-based customized image. Sailfish has its own Android AppSupport, but that requires a Sailfish license, does not support the Play Store client or Google Play Services, and Jolla explicitly says it cannot guarantee 100% Android app compatibility. [20]

The official Google side of the story is even blunter. The Play Integrity API is designed to help apps decide whether they are running on a **genuine certified Android device**. Google’s own help pages say Play Protect certification is tied to compatibility testing and licensing, and they also say devices with an unlocked bootloader or modified custom Android OS may fail certification checks. In other words, once you leave stock Android compatibility land, app acceptance becomes a policy question controlled by the app developer, not by you. [21]

Even the “Android on Linux” bridge is not a free pass. Waydroid’s own certification docs say a GAPPS-based Waydroid setup starts by telling you the device is **not certified for Google Play Protect**. That alone does not prove every banking app will fail, but combined with Play Integrity it explains why financial apps, national ID apps, contactless payment apps, and anti-fraud-heavy apps are the first places things blow up. [22]

GrapheneOS is useful here as a reality check because it is still Android-compatible and yet still has trouble with some financial apps. Its usage guide says banking apps are a particularly problematic class for alternate operating systems and notes increasing use of attestation and certification checks. Its attestation guide also shows the more hopeful side: developers **could** support secure alternate

operating systems via the hardware attestation API instead of brute-force banning anything that is not stock. But “could” is not the same as “will.” [23]

So the honest conclusion is this: **if banking/ID/payment apps are a hard requirement, a Linux-only phone OS is still a gamble in 2026**. The least painful route remains an **AOSP-compatible** aftermarket OS with a modern Android app model, even though attestation and certification can still bite there too. Google’s compatibility, kernel, verified boot, and OTA docs show why that path remains the best base if you care about sensitive Android apps: you keep the Android application environment and the update/security plumbing that those apps were designed around. [24]

A practical build strategy

The strategic move is **not** “support every Android phone.” It is “pick one friendly hardware family and do it properly.” The best targets are phones with unlockable bootloaders, published flashing/build guidance, and SoCs with decent upstream chances. The postmarketOS community specifically calls Qualcomm SoCs generally good candidates for mainline Linux, and Sony’s Open Devices program provides official unlocking, flashing, AOSP build, and kernel-compilation guidance for Xperia hardware. That combination is exactly what you want when building a serious replacement OS instead of a weekend science fair. [25]

For the actual engineering path, the winning pattern is hybrid. **Phase one:** get the phone booting and talking to hardware via Halium-style reuse of the vendor kernel, Android daemons, and libhybris. **Phase two:** run a Linux userspace you actually want to maintain long term, usually Debian-style or Alpine-style, plus a shell such as Phosh, Lomiri, or Plasma Mobile. **Phase three:** upstream and mainline aggressively so you can replace old vendor kernels and Android-only bits with generic kernels and normal Linux maintenance over time. postmarketOS’s recent work on generic kernel packages is exactly the sort of move that turns a pile of ports into a future platform. [26]

That leads to one blunt recommendation. **If you want the fastest path to a usable Linux phone on existing Android hardware, start from Droidian or Ubuntu Touch and steal every lesson you can from Halium. If you want the best long-term post-Android future, aim toward postmarketOS-style mainline Linux as the end state. If you want banking apps with minimal pain, keep an AOSP-compatible branch — or accept that your Linux phone and your banking/ID device may need to be different things.** [27]

Bottom line

Your idea is technically sound, but the devil is gloriously annoying. **Yes, you can build a new Linux-based phone OS on top of existing Android phones. No, you usually cannot do it as a pure from-scratch, all-open stack without massive pain.** The short-term road is reuse; the long-term road is mainlining. The truly hard part is not building the launcher, package manager, or app store. It is taming display, touch, modem, Wi-Fi, audio, camera, power, boot, and updates on hardware that was never designed for an open general-purpose Linux stack. [28]

If you force me to compress the whole report into one answer: **for current Android handsets, the best practical build bases are Droidian and Ubuntu Touch for bring-up, postmarketOS for the long game, Sailfish OS if you want a more polished but less open reference, and Mobian as the clean Debian blueprint. For banking apps, the honest winner is still an AOSP-compatible OS, not a pure Linux phone.** Existing Android phones are not going to become dead bricks in September 2026; what

gets worse is your independence from Google's ecosystem and from vendor-era Android plumbing unless you deliberately build around that reality. [29]

Open questions and limits

The biggest remaining uncertainty is **device-specific viability**. Whether this is pleasantly hackable or soul-destroying depends heavily on the exact phone model, whether the bootloader is unlockable, the SoC family, carrier/IMS requirements, camera expectations, and whether NFC and banking apps are mandatory. Sony's official unlock docs, Ubuntu Touch's device pages, and the Linux-phone projects' own support matrices all show that support quality varies wildly by device, even inside the same OS family. [30]

The least direct fit among the five is Mobian on random Android phones: its documentation is clearest as a Debian-mobile reference and as a Linux-first-device target, not as a turn-key Android-handset port base. So if the real mission is "turn ordinary old Android phones into our own Linux phones," Droidian/Halium and postmarketOS are the two most strategic poles to think between. [31]

---

[1] [5] [7] [17] [28] Introduction - UBports documentation
https://docs.ubports.com/en/latest/porting/introduction/index.html

[2] Site updates | Android Open Source Project
https://source.android.com/docs/whatsnew/site-updates

[3] [29] Android Developers Blog: A new layer of security for certified Android devices
https://android-developers.googleblog.com/2025/08/elevating-android-security.html

[4] Android Security Bulletin—May 2026 | Android Open Source Project
https://source.android.com/docs/security/bulletin/2026/2026-05-01

[6] [11] [13] [27] FAQ | Droidian
https://docs.droidian.org/faq/

[8] [26] Halium Project
https://halium.org/

[9] Porting Guide - WebOS-Ports
https://webos-ports.org/wiki/Porting_Guide

[10] Generic Kernel Image (GKI) project | Android Open Source Project
https://source.android.com/docs/core/architecture/kernel/generic-kernel-image

[12] Plasma Mobile
https://plasma-mobile.org/

[14] postmarketOS // real Linux distribution for phones
https://postmarketos.org/

[15] Ubuntu Touch • Linux mobile OS that gives you pure freedom
https://devices.ubuntu-touch.io/

[16] [31] Mobian/Devices
https://wiki.debian.org/Mobian/Devices?utm_source=chatgpt.com

[18] Supported Devices | Sailfish OS Documentation
https://docs.sailfishos.org/Support/Supported_Devices/

19 Mobian
https://mobian-project.org/

20 Android apps
https://docs.ubports.com/en/latest/userguide/dailyuse/waydroid.html?utm_source=chatgpt.com

21 Overview of the Play Integrity API | Android Developers
https://developer.android.com/google/play/integrity/overview

22 Google Play Certification | Waydroid
https://docs.waydro.id/faq/google-play-certification

23 Usage guide | GrapheneOS
https://grapheneos.org/usage

24 Android Compatibility program overview | Android Open Source Project
https://source.android.com/docs/compatibility/overview

25 Qualcomm mainline porting
https://wiki.postmarketos.org/wiki/Qualcomm_mainline_porting?utm_source=chatgpt.com

30 Unlock bootloader | Developer World
https://developer.sony.com/open-source/aosp-on-xperia-open-devices/get-started/unlock-bootloader